

NovaMart — Detailed Data Analyst Reference & Cheat Sheet

1. End-to-End NovaMart Workflow

Use this as the detailed reference for reproducing the project and explaining the workflow in interviews.

Business understanding

- Define the business question and decision to support.
- Identify stakeholders, required metrics, time period and granularity.
- Separate executive questions from diagnostic questions.

Data inventory

- List every table/file and identify its business meaning.
- Determine the grain of each table (what one row represents).
- Identify key fields and relationships.

Profiling

- Inspect shape, columns, dtypes, nulls, duplicates and unique values.
- Profile numeric ranges and distributions.
- Inspect date ranges and categorical values.

Validation

- Check identifier uniqueness.
- Check relationships between related tables.
- Check impossible values, unexpected zeros and inconsistent categories.
- Reconcile important totals before analysis.

Cleaning

- Standardize data types and text values.
- Handle missing values according to business meaning.
- Remove true duplicates only after validation.
- Preserve raw data separately.

Transformation

- Create analytical measures such as Sales, Cost, Profit and Profit Margin.
- Create useful date dimensions and business groupings.
- Prepare analysis-ready tables without losing source traceability.

Analysis

- Start with overall KPIs.

- Compare time trends, regions, products, customers and sellers.
- Separate volume metrics from efficiency metrics.
- Investigate high-sales versus high-margin differences.

Visualization

- Use KPI cards for headline metrics.
- Use line charts for trends.
- Use bars for comparisons and rankings.
- Use stacked charts for composition.
- Use scrolling when dimensions have many members.

Dashboard QA

- Verify filters, calculations, sorting, labels, tooltips and number formats.
- Cross-check dashboard totals against source calculations.
- Check clipping, spacing and redundant visual elements.

Communication

- State what happened, where, why it matters, and potential actions.
- Document assumptions, limitations and metric definitions.

2. NovaMart Data Model

Table	Rows	Purpose	Important fields
Customers	2,000	Customer master	Customer_ID, Segment, Region, Signup_Date
Products	420	Product master	Product_ID, Category, Subcategory, Unit_Cost, Unit_Price
Sellers	100	Seller master	Seller_ID, Seller_Type, Seller_Region
Orders	50,000	Order header	Order_ID, Order_Date, Customer_ID, Seller_ID, Status, Payment, Shipping
Order Items	105,953	Transactional detail	Order_ID, Product_ID, Seller_ID, Quantity, Discount, Sales, Cost, Profit

3. Data Quality Checklist

- Shape: `df.shape`
- Columns and types: `df.columns`, `df.dtypes`, `df.info()`
- Nulls: `df.isna().sum()`
- Duplicates: `df.duplicated().sum()`
- Unique values: `df[col].nunique()`, `df[col].unique()`
- Numeric profile: `df.describe()`
- Date profile: min/max date and unique dates
- Identifier checks: uniqueness and nullability
- Relationship checks: unmatched foreign keys after joins

- Business checks: Sales ≥ 0 where appropriate, Cost/Profit logic, valid percentages and valid status categories

4. Python / Pandas Detailed Reference

Import libraries

```
import pandas as pd
import numpy as np
```

Core data-analysis libraries.

Load data

```
df = pd.read_csv('file.csv')
```

Load a CSV into a DataFrame.

Inspect first rows

```
df.head()
df.tail()
```

Quick visual inspection.

Shape

```
df.shape
```

Rows and columns.

Column list

```
df.columns
```

Available fields.

Data types

```
df.dtypes
df.info()
```

Check types and non-null counts.

Summary statistics

```
df.describe(include='all')
```

Profile numeric and categorical fields.

Null counts

```
df.isna().sum()
```

Find missing values by column.

Duplicate count

```
df.duplicated().sum()
```

Find duplicate rows.

Unique values

```
df['Region'].unique()
df['Region'].nunique()
```

Inspect categories.

Filter rows

```
df[df['Sales'] > 100000]
```

Single-condition filtering.

Multiple conditions

```
df[(df['Region']=='East') & (df['Profit']>100000)]
```

Compound filtering.

Select columns

```
df[['Order_ID', 'Sales', 'Profit']]
```

Keep required fields.

Sort

```
df.sort_values('Sales', ascending=False)
```

Rank records.

Group and sum

```
df.groupby('Category')['Sales'].sum()
```

Category-level totals.

Multiple aggregations

```
df.groupby('Category').agg(Sales=('Sales','sum'), Profit=('Profit','sum'))
```

Build summary tables.

Calculated metric

```
df['Profit_Margin'] = df['Profit'] / df['Sales'] * 100
```

Create a percentage metric.

Date conversion

```
df['Order_Date'] = pd.to_datetime(df['Order_Date'])
```

Convert date fields.

Date components

```
df['Year'] = df['Order_Date'].dt.year  
df['Month'] = df['Order_Date'].dt.month
```

Create time dimensions.

Rename columns

```
df.rename(columns={'old':'new'}, inplace=True)
```

Standardize names.

Drop duplicates

```
df = df.drop_duplicates()
```

Remove confirmed duplicate rows.

Merge tables

```
df = orders.merge(customers, on='Customer_ID', how='left')
```

Combine related tables.

Concat

```
df = pd.concat([df1, df2], ignore_index=True)
```

Append compatible tables.

Value counts

```
df['Payment_Method'].value_counts()
```

Frequency of categories.

Replace values

```
df['Status'] = df['Status'].replace({'Complete': 'Completed'})
```

Standardize labels.

Export

```
df.to_csv('cleaned_data.csv', index=False)
```

Save analysis-ready data.

5. SQL Detailed Reference

SELECT

```
SELECT Customer_ID, Sales FROM order_items;
```

Choose columns.

WHERE

```
SELECT * FROM order_items WHERE Profit > 100000;
```

Filter rows.

ORDER BY

```
SELECT * FROM order_items ORDER BY Profit DESC;
```

Sort results.

DISTINCT

```
SELECT DISTINCT Payment_Method FROM orders;
```

Find unique categories.

COUNT

```
SELECT COUNT(*) FROM orders;
```

Count rows.

SUM / AVG

```
SELECT SUM(Sales), AVG(Profit) FROM order_items;
```

Aggregate metrics.

GROUP BY

```
SELECT Category, SUM(Sales) AS Sales FROM order_items GROUP BY Category;
```

Create grouped summaries.

HAVING

```
SELECT Category, SUM(Sales) AS Sales FROM order_items GROUP BY Category HAVING  
SUM(Sales)>1000000;
```

Filter grouped results.

CASE

```
SELECT CASE WHEN Profit>0 THEN 'Profitable' ELSE 'Loss' END AS Profit_Status FROM order_items;
```

Create business categories.

INNER JOIN

```
SELECT * FROM customers c INNER JOIN orders o ON c.Customer_ID=o.Customer_ID;
```

Keep matching rows.

LEFT JOIN

```
SELECT * FROM customers c LEFT JOIN orders o ON c.Customer_ID=o.Customer_ID;
```

Keep all rows from the left table.

CTE

```
WITH sales AS (SELECT Category, SUM(Sales) AS total_sales FROM order_items GROUP BY Category)  
SELECT * FROM sales;
```

Break complex queries into readable steps.

Window rank

```
SELECT Product_ID, Sales, RANK() OVER (ORDER BY Sales DESC) AS sales_rank FROM product_sales;
```

Rank without collapsing rows.

Window partition

```
SELECT Seller_ID, Sales, RANK() OVER (PARTITION BY Seller_Region ORDER BY Sales DESC) AS  
region_rank FROM seller_sales;
```

Rank within groups.

6. Metric Definitions

Sales

Primary revenue measure used for overall performance and volume comparisons.

Cost

Associated cost measure from transactional detail.

Profit

Sales minus Cost.

Profit Margin

Profit divided by Sales, expressed as a percentage. Validate zero-sales cases before calculating.

Top Products by Sales

Products ranked by total Sales, not profit.

Top Products by Profit Margin

Products ranked by Profit Margin. Consider minimum-sales thresholds in real business analysis to avoid misleading tiny-volume products.

7. Tableau Build & QA Reference

- Build worksheets around one business question each.
- Use Text marks for headline KPI numbers.
- Use bars for category comparisons and rankings.
- Use line charts for trends.
- Use filters when users need controlled exploration.
- Use tiled containers for predictable layouts.
- Use compact number formats such as M/B for large Sales and Profit values.
- Keep percentage metrics at one decimal place where appropriate.
- Use tooltips for secondary detail rather than overcrowding marks.
- Use scrolling for high-cardinality seller/product charts.
- Perform a final cross-dashboard consistency check.

8. NovaMart Dashboard Map

Dashboard 1 — Executive Overview

Total Profit; Total Sales; Profit Margin; Monthly Sales & Profit Trend; Sales by Region; Sales by Product Category.

Dashboard 2 — Detailed Performance

Customer segment; product subcategory; seller; seller region; order/payment; shipping/order status analysis.

Dashboard 3 — Profitability & Top Performers

Profit Margin Trend; Profit by Product Category; Profit by Product Subcategory; Top Products by Sales; Top Products by Profit Margin; Sales by Seller Type; Profit Margin by Seller Region.

9. Interview Explanation Framework

A concise way to explain NovaMart: “I worked on a multi-table retail analytics project where I first profiled and validated the data, prepared analysis-ready datasets, calculated sales/profitability metrics, used Python/Pandas and SQL for analysis, and built three Tableau dashboards. I separated the dashboards into executive overview, detailed performance, and profitability/top performers so each served a distinct business purpose. I then cross-checked the dashboard outputs and completed a final QA pass.”

10. Reusable Project Completion Checklist

- Business question documented
- Data model understood
- Raw data preserved
- Data profiled
- Quality issues documented and resolved
- Metrics defined
- Analysis completed
- Visualizations validated
- Dashboard layout polished
- Dashboard QA completed
- Project summary written
- Workflow documented
- Code/syntax reference prepared
- Key insights documented
- Portfolio/interview explanation prepared